



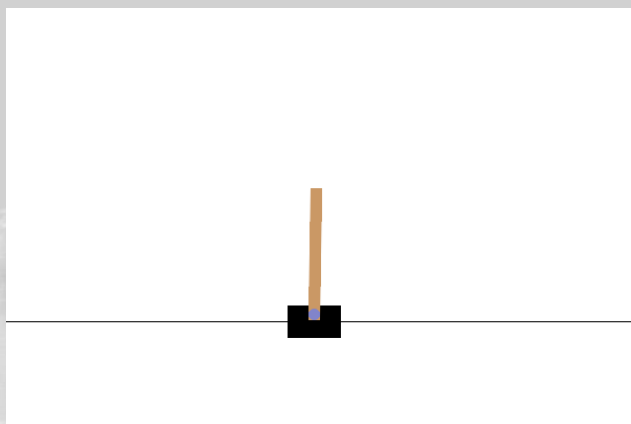
北京大学
PEKING UNIVERSITY

DQN作业介绍

鲁云龙 2025.11.6

作业要求

- 在CartPole和Breakout环境上实现DQN算法，在提供的示例代码基础上进行补全、修改，提交实验报告。





实验环境

- Gym Atari环境
 - CartPole-v0
 - 状态空间4
 - 动作空间2（向左、向右）
 - BreakoutDeterministic-v4
 - 状态空间 $210 \times 160 \times 3$ ，图像输入
 - 动作空间4（无操作、发射球、挡板向左、挡板向右）



示例代码

- 提供一份标准化的训练框架代码
 - 将RL环境、RL算法、神经网络模型、训练代码分别组件化
 - 同一份算法代码可以不做修改直接适配不同的环境
- 推荐作业步骤：
 - (1) 将DQN算法代码中空缺的部分补全
 - (2) 在CartPole环境上尝试调整网络结构和训练参数，对比训练效果
 - (3) 适配Breakout环境训练，修改Q网络结构和环境部分代码
 - (4) 在Breakout环境上尝试调整网络结构和训练参数，对比训练效果

示例代码结构

- env.py
 - Env类定义通用的RL环境接口
- gym_env.py
 - GymEnv类对通用Gym环境包装
 - BreakoutEnv类对Breakout环境包装，增加了图像预处理提高训练效率

```
3  # RL环境基类
4  class Env(ABC):
5
6      # 开始新的episode
7      @abstractmethod
8      def reset(self, conf):
9  >      ''' ...
12
13      pass
14
15      # 单步状态转移
16      @abstractmethod
17  >      def step(self, action):
18          ''' ...
20
21      pass
```



示例代码结构

- model.py
 - Model类定义通用的模型接口
- q_network.py
 - QNetwork类实现了针对CartPole的简易Q网络

```
3  # 模型基类
4  class Model(ABC):
5
6      # 模型推理（前向传播）
7      @abstractmethod
8      def inference(self, obs):
9          pass
10
11     # 模型训练（反向传播）
12     @abstractmethod
13     def train(self, loss):
14         pass
```



示例代码结构

- agent.py
 - Agent类定义通用的算法接口
- dqn_agent.py
 - DQNAgent类实现DQN算法（需要补全）

```
3  # RL算法基类
4  class Agent(ABC):
5
6      # 训练时采样动作
7      @abstractmethod
8      def predict(self, obs):
9          pass
10
11     # 测试时选最优动作
12     @abstractmethod
13     def exploit(self, obs):
14         pass
15
16     # 从一批样本中训练
17     @abstractmethod
18     def learn(self, samples):
19         pass
20
21     # 算法相关的样本预处理
22     @abstractmethod
23     def sample_process(self, samples):
24         pass
```



示例代码结构

- dqn_train.py
 - 训练流程
 - 将环境、算法、模型组件作为黑箱调用

```
replay_buffer = deque(maxlen = buffer_size) # 样本池
for episode in tqdm(range(epochs)):
    ret = 0
    obs = env.reset()
    done = False
    while not done:
        action = agent.predict(obs)
        next_obs, reward, done, _ = env.step(action)
        ret += reward
        sample = FrameNumpy.from_dict({
            'obs': obs,
            'next_obs': next_obs,
            'action': action,
            'reward': reward,
            'done': done
        })
        obs = next_obs
        # 每个step产生的样本加入样本池，并直接采样batch进行单次训练
        replay_buffer.append(sample)
        if len(replay_buffer) > batch_size:
            batch = random.sample(replay_buffer, batch_size)
            batch = SampleBatchNumpy.stack(batch)
            agent.sample_process(batch)
            agent.learn(batch)
```




示例代码结构

- sample.py
 - Frame类，对单帧观测/样本的抽象
 - SampleBatch类，对样本Batch的抽象
 - 细节比较复杂，为了保证算法代码在面向不同观测空间时的可复用性



框架通用性

- 针对不同的环境（状态空间、动作空间），同一份算法代码可以完全复用
- Gym环境的状态空间基本都是简单的numpy数组
obs: `numpy.ndarray(shape=N)`
- 麻将的状态空间包含特征和可行动作掩码两部分numpy数组
obs: `dict(state=numpy.ndarray(shape=174*4*9),
action_mask=numpy.ndarray(shape=235))`
- 难点：算法的各个函数都需要能兼容多字段的状态

框架通用性

- 样本类Frame和SampleBatch支持多层key-value的字典
- 例如DQN算法中，训练所需的抽象字段为obs, action, next_obs等，但obs可以包含action_mask等多个子字段
- 算法不关注obs内部结构，而是直接喂给处理不同输入模型

```
# 输入样本Batch, 训练模型
def learn(self, samples: SampleBatchNumpy):
    ...

    samples需要包含字段: obs, next_obs, action, reward, done
    ...

    samples = samples.to_torch(device = self.device)
    # 计算Q(s, a)
    q_values = self.model.inference(samples.obs)
    q_value = ... # 此处需补全代码
    # 计算Q(s', a)
    next_q_values = self.model.inference(samples.next_obs)
    max_next_q_value = ... # 此处需补全代码
    # Q函数更新目标
    q_target = ... # 此处需补全代码
    loss = ... # 此处需补全代码
    # 模型更新
    self.model.train(loss)
```



框架通用性

- 举例：CartPole的样本

- `samples.obs`: $B*4$
- `samples.action`: $B*1$
- `samples.reward`: $B*1$
- `samples.done`: $B*1$
- `samples.next_obs`: $B*4$

- 举例：麻将的样本

- `samples.obs.state`: $B*174*4*9$
- `samples.obs.action_mask`: $B*235$
- `samples.action`: $B*1$
- `samples.reward`: $B*1$
- `samples.done`: $B*1$
- `samples.next_obs.state`: $B*174*4*9$
- `samples.next_obs.action_mask`: $B*235$

框架通用性

- SampleBatch内部实现是将多层字典摊平成一个字典
 - 例如samples.obs.state实际对应一个“obs.state”的key
 - 重载了set_attr/get_attr和set_item/get_item，同时支持samples.obs和samples['obs']等使用方式，获取往里一层的字典
 - 实现了to_torch函数，将每个键的numpy数组都转成torch数组
 - 实现了offset工具函数，支持对子字段进行时间步的移位，整体构造新字段
- 例如PPO算法的样本处理，将一整局obs整体移位得到next_obs

```
def sample_process(self, samples: SampleBatchNumpy) -> None:
    samples.next_obs = samples.obs.offset(-1)
    samples.action = samples.action.reshape((-1, 1)).astype(np.int64)
    samples.reward = samples.reward.reshape((-1, 1)).astype(np.float32)
    samples.done = samples.done.reshape((-1, 1)).astype(np.float32)
```

框架通用性

- 对于单帧的状态，使用Frame类进行抽象
 - 既支持普通的numpy数组（FrameArrayNumpy类）
 - 也支持复杂的字典（FrameDict类）
 - 均实现了to_torch函数，可以转成torch tensor进行模型推理/训练
- 算法实现中，predict和exploit输入状态直接转成Frame
 - 不用管内部结构是普通数组还是复杂字典
 - 不同环境的训练不用改算法代码

```
# 输入单帧状态，采样探索性动作
def predict(self, obs: 'Frame | np.ndarray | dict'):
    obs = Frame.convert(obs)
    if np.random.random() < self.epsilon:
        # 以  $\epsilon$  的概率随机选择动作
        action = ... # 此处需补全代码
    else:
        # 以  $1-\epsilon$  的概率选择Q值最大的动作
        obs = obs.to_torch(device = self.device)
        q_value = self.model.inference(obs)
        action = ... # 此处需补全代码
    return action
```

框架通用性

- 由于不同环境状态空间的区别，全都隐藏到了obs内部结构中
- 训练流程中收集样本的代码也比较固定

```
replay_buffer = deque(maxlen = buffer_size) # 样本池
for episode in tqdm(range(episodes)):
    ret = 0
    obs = env.reset()
    done = False
    while not done:
        action = agent.predict(obs)
        next_obs, reward, done, _ = env.step(action)
        ret += reward
        sample = FrameNumpy.from_dict({
            'obs': obs,
            'next_obs': next_obs,
            'action': action,
            'reward': reward,
            'done': done
        })
        obs = next_obs
        # 每个step产生的样本加入样本池，并直接采样batch进行单次训练
        replay_buffer.append(sample)
        if len(replay_buffer) > batch_size:
            batch = random.sample(replay_buffer, batch_size)
            batch = SampleBatchNumpy.stack(batch)
            agent.sample_process(batch)
            agent.learn(batch)
```